Sign in

pytorch / pytorch Public

Code Issues 5k+

Commit da0fad8



colesbury authored and soumith committed on Jun 30, 2017

Use torch.matmul in nn.Linear (#1935)

```
This takes advantage of the broadcasting behavior of
torch.matmul to
support inputs with more than two dimensions. The extra
dimensions are
treated like part of the batch dimension, much like
nn.Bottle in Lua
Torch.

There are a few related small performance changes:

 * Addmm computes the gradient in column-major for inputs
in
   column-major format
 * Variable.mm calls Addmm in-place with the desired output
buffer
```

whc_flight_4 ...

main (#1935) · bc2caa7fdf006894eff7af936babde69ab5a40f8-huydhn-debug

1 parent 2c038f2 commit da0fad8

7 files changed +29 -46 lines changed

Search within code

+6 ••• test/test_nn.py

```
@@ -2316,6 +2316,12 @@ def test_upsamplingTrilinear3d(self):
2316  2316          input = Variable(torch.randn(1, 1, 2, 2, 2), requires_grad=True)
2317  2317
                    self.assertTrue(gradcheck(lambda x: F.upsample(x, 4, mode='trilinear'), (input,)))
2318  2318
      2319  +
```

```
                def test_linear_broadcasting(self):
2320 +                  m = nn.Linear(5, 8)
2321 +                  inp = Variable(torch.randn(2, 3, 5))
2322 +                  expected = m(inp.view(6, 5)).view(2, 3, 8)
2323 +
                        self.assertEqual(expected, m(inp))
2324 +
2319 2325          def test_bilinear(self):
2320 2326              module = nn.Bilinear(10, 10, 8)
2321 2327              module_legacy = legacy.Bilinear(10, 10, 8)
```





+10 -2 

torch/autograd/_functions/blas.py

```
@@ -36,12 +36,20 @@ def backward(ctx, grad_output):
 
             grad_add_matrix = grad_add_matrix.mul(ctx.alpha)
 
-            if ctx.needs_input_grad[1]:
-                grad_matrix1 = torch.mm(grad_output, matrix2.t())
+            if ctx.needs_input_grad[1]:
+                if matrix1.stride() == (1, matrix1.size(0)):
+                    # column major gradient if input is column major
+                    grad_matrix1 = torch.mm(matrix2, grad_output.t()).t()
+                else:
```

```
                43 +                grad_matrix1 =
                   torch.mm(grad_output,
                   matrix2.t())
40              44                  if ctx.beta != 1:
41              45                      grad_matrix1
                   *= ctx.beta
42              46
43              47              if
                   ctx.needs_input_grad[2]:
44                  -               grad_matrix2 =
                   torch.mm(matrix1.t(),
                   grad_output)
                48 +                if
                   matrix2.stride() == (1,
                   matrix2.size(0)):
                49 +                    # column major
                   gradient if input is column
                   major
                50 +                    grad_matrix2 =
                   torch.mm(grad_output.t(),
                   matrix1).t()
                51 +                else:
                52 +
```

```
                        grad_matrix2 =
    torch.mm(matrix1.t(),
    grad_output)
45  53                  if ctx.beta != 1:
46  54                      grad_matrix2
        *= ctx.beta
47  55
```

+1 -1

torch/autograd/variable.py

```
    @@ -544,7 +544,7 @@ def
    _blas(self, cls, args,
    inplace):
544 544
545 545          def mm(self, matrix):
546 546              output =
        Variable(self.data.new(self.d
        ata.size(0),
        matrix.data.size(1)))
547     -            return
        self._static_blas(Addmm,
```

```
                (output, 0, 1, self, matrix),
                False)
547 +           return
                Addmm.apply(output, self,
                matrix, 0, 1, True)
548 548
549 549       def bmm(self, batch):
550 550           output =
                Variable(self.data.new(self.d
                ata.size(0),
                self.data.size(1),
```





-30 



torch/nn/_functions/linear.py 

```
...   @@ -1,35 +1,5 @@
1   1   import torch
2   2   from torch.autograd import
        Function
3       - from torch.autograd import
        Variable
4       -
```

```
-
- class Linear(Function):
-
-     @staticmethod
-     def forward(ctx, input, weight, bias=None):
-         ctx.save_for_backward(input, weight, bias)
-         output = input.new(input.size(0), weight.size(0))
-         output.addmm_(0, 1, input, weight.t())
-         if bias is not None:
-             output.add_(bias.expand_as(output))
-         return output
-
-     @staticmethod
-
```

```
                def backward(ctx,
            grad_output):
19          -           input, weight, bias =
            ctx.saved_variables
20          -
21          -           grad_input =
            grad_weight = grad_bias = None
22          -           if
            ctx.needs_input_grad[0]:
23          -               grad_input =
            torch.mm(grad_output, weight)
24          -           if
            ctx.needs_input_grad[1]:
25          -               grad_weight =
            torch.mm(grad_output.t(),
            input)
26          -           if bias is not None
            and ctx.needs_input_grad[2]:
27          -               grad_bias =
            grad_output.sum(0, False)
28          -
29          -           if bias is not None:
30          -
```

```
                return grad_input,
    grad_weight, grad_bias
-           else:
-               return grad_input,
    grad_weight


class Bilinear(Function):
```

-2

torch/nn/backends/thnn.py

```
@@ -20,7 +20,6 @@ def
_get_thnn_function_backend():


    def _initialize_backend():
        from .._functions.thnn
    import _all_functions as
    _thnn_functions
-       from .._functions.linear
    import Linear
```

```
24  23          from ._functions.conv
    import ConvNd
25  24          from ._functions.rnn
    import RNN, \
26  25                  RNNTanhCell,
    RNNReLUCell, GRUCell, LSTMCell
    @@ -29,7 +28,6 @@ def
    _initialize_backend():
29  28          from ._functions.loss
    import CosineEmbeddingLoss, \
30  29                  HingeEmbeddingLoss,
    MarginRankingLoss
31  30
32      -
        backend.register_function('Lin
        ear', Linear)
33  31
        backend.register_function('Con
        vNd', ConvNd)
34  32
        backend.register_function('RNN
        ', RNN)
35  33
```

```
backend.register_function('RNN
TanhCell', RNNTanhCell)
```

+11 -7

torch/nn/functional.py

```
@@ -7,6 +7,7 @@
 import torch
 from . import _functions
 from .modules import utils
+from ._functions.linear import Bilinear
 from ._functions.padding import ConstantPad2d
 from ..autograd import _functions as _autograd_functions
 from torch.autograd import Variable
@@ -532,18 +533,21 @@ def sigmoid(input):
```

```
532  533      # etc.
533  534
534  535      def linear(input, weight,
              bias=None):
535       -      if bias is None:
536       -          return
              _functions.linear.Linear.appl
              y(input, weight)
537       -      else:
538       -          return
              _functions.linear.Linear.appl
              y(input, weight, bias)
     536  +      if input.dim() == 2 and
              bias is not None:
     537  +          # fused op is
              marginally faster
     538  +          return
              torch.addmm(bias, input,
              weight.t())
     539  +
     540  +      output =
              input.matmul(weight.t())
     541  +      if bias is not None:
```

```
                +         output += bias
                +     return output
539 544
540 545
541 546     def bilinear(input1, input2,
            weight, bias=None):
542         -     state =
            _functions.linear.Bilinear()
543 547         if bias is None:
544         -         return state(input1,
            input2, weight)
            548 +         return Bilinear()
            (input1, input2, weight)
545 549         else:
546         -         return state(input1,
            input2, weight, bias)
            550 +         return Bilinear()
            (input1, input2, weight,
            bias)
547 551
548 552
549 553
```

```
def batch_norm(input,
running_mean, running_var,
weight=None, bias=None,
```

+1 -4

torch/nn/modules/linear.py

```
@@ -48,10 +48,7 @@ def
reset_parameters(self):
48  48          self.bias.data.uniform_(-stdv,
                stdv)
49  49
50  50      def forward(self, input):
51      -         if self.bias is None:
52      -             return
              self._backend.Linear.apply(inp
              ut, self.weight)
53      -         else:
54      -             return
              self._backend.Linear.apply(inp
              ut, self.weight, self.bias)
```

```
51  +           return F.linear(input,
    self.weight, self.bias)
55  52
56  53          def __repr__(self):
57  54              return
            self.__class__.__name__ + ' ('
            \
```



Comments 0 (0)



Please sign in to comment.